

AI Blue Team Response

Lesson 2 of 2 — Implement, Retest, and Prove It's Fixed

The red team's engagement report is in. NorthPeak Credit Union's leadership wants the findings fixed before Piper launches — and they want proof, not just a promise, that it's actually fixed.

The Assignment

— *Follow-up email from NorthPeak Credit Union* —

"Thanks for the assessment. Leadership wants these findings fixed before we launch — and they want to see evidence it actually worked, not just a note saying it's handled."

Each student/pair receives a set of red-team findings to remediate — either the report they wrote in Lesson 1, a classmate's report via peer-swap, or the reference findings sheet below. Every finding you received was found against hardened_bot with the gateway already on (Condition 3) — meaning the prompt and the default gateway rules together were not enough to stop it. You have two levers available to close each one, and you choose which to pull.

Reference Findings Sheet (for standalone use, or as a common baseline)

Essential Question

How do you turn a security finding into an actual fix, decide which defense layer to fix it in, and prove the fix works?

Learning Objectives

- **Apply** — For each finding, choose and implement the appropriate remediation lever: the model's own instructions (editing a Modelfile and rebuilding), the security gateway's filter rules (filter_rules.py), or both — to address specific findings from a red-team engagement report.
- **Evaluate** — Re-run the original attack set against the implemented defenses and evaluate whether each finding was fully remediated, partially mitigated, or still open, using empirical retest evidence.
- **Create / Analyze (core)** — Design, implement, and justify at least one new filter rule not provided out of the box, closing a gap identified in retesting.

Materials & Setup

Instructor pre-class checklist

- Collect and redistribute Lesson 1 reports (peer-swap) before class, or have the reference findings sheet ready if teaching this as a stand-alone lesson.
- The Docker stack from Lesson 1 should still be running (docker compose up -d), or rebuilt from the same Modelfiles.
- Test the gateway's default rule set end-to-end yourself before class so you know exactly which findings are already closed and which are your discussion anchor — the defaults are deliberately incomplete (see Instructor Notes).

Setup commands

```
# If the stack isn't already running from Lesson 1:  
docker compose up -d
```

```
docker compose exec llm ollama create vulnerable_bot -f
/app/lab/modelfiles/vulnerable.txt
docker compose exec llm ollama create hardened_bot -f /app/lab/modelfiles/hardened.txt

# Open the gateway
http://localhost:5000
```

The gateway is already running as part of the stack — there's no separate script to start. All remediation and retesting for this lesson happens through this same browser page.

75–90-Minute Session Timeline

Implementation Instructions

For each finding you received, decide which lever closes it, implement or confirm that fix, then retest the exact attack that produced the original finding.

Choosing your remediation lever

- **Model instructions** — edit lab/modelfiles/hardened.txt to close a behavioral gap, then rebuild: `docker compose exec llm ollama create hardened_bot -f /app/lab/modelfiles/hardened.txt`. Affects everything the model does, but takes a rebuild to test each change.
- **Filter rules** — edit lab/scripts/filter_rules.py. The gateway hot-reloads this file on every request, so save and refresh the browser to retest immediately — no rebuild, no restart.
- **Both** — the strongest fix for a finding is usually defense-in-depth: narrow the model's willingness to comply and back it up with a filter rule in case the instruction alone gets bypassed by different phrasing.

Record which lever (or combination) you used for each finding in the Before/After Validation Matrix — this is graded as part of remediation mapping.

Before/After Validation Matrix

Custom Filter Rule (core requirement)

Design one new input or output filter rule not already provided in filter_rules.py. State which finding or gap it closes and explain the specific risk it mitigates.

```
# Example structure – edit lab/scripts/filter_rules.py
Add to INGRESS_BLACKLIST: "<trigger phrase you observed>"
    or
Add to EGRESS_SECRETS / EGRESS_PATTERNS: "<leaked string or phrase you observed>"

# Justification: this closes finding ____ because ____
```

The gateway currently only supports block — there's no partial "flag" or "rewrite" action. If your gap needs something more nuanced than a block, say so in your justification; identifying the limitation is itself a valid finding.

Implementation & Validation Report (Summative Assessment)

- Written report (findings-to-fix mapping + retest evidence + custom rule justification), OR
- Recorded "remediation readout" video (2–3 minutes) walking through what you fixed and how you proved it.

Close with a short Residual Risk note: what, if anything, is still not fully covered after your implementation — a real remediation report never claims 100% risk elimination.

Rubric

Instructor Notes

Expected result: the default filter rules are deliberately incomplete, not a finished product — treat that as the premise of the lesson, not a bug to apologize for. In our own testing, Obfuscation/Encoding is the category most likely to still leak under the out-of-the-box rules, because the default checks look for exact secret strings and phrases, not disguised or paraphrased ones. If a group's retest still shows a leak anywhere, that's a genuine and useful teaching moment — walk through their `filter_rules.py` or `Modelfile` edit live rather than just correcting it for them. Worth naming explicitly if it comes up: this is a structural limit of string/regex matching, not a tuning problem you can eventually solve by adding enough phrases. Every new rule a group writes only catches the exact wording it was written for — a determined attacker just rephrases. This is exactly the gap that context-aware policy (Phase 3 / OPA, previewed at the end of the webinar) is designed to close, by classifying intent instead of matching text. Students don't need to know Phase 3 exists to learn this lesson, but if a sharp group asks "couldn't we just keep adding rules forever," this is the honest answer.

Common shortfall to watch for: students sometimes treat "I added a rule" as equivalent to "I proved the rule works." Hold the retest-evidence line — a config change without a retest doesn't earn full credit on this rubric. Watch too for over-broad rules that technically pass every red-team prompt but would also block a legitimate customer question — ask each group to sanity-check their final rules against 1–2 benign prompts before they call a finding closed.

If running Lesson 2 as a stand-alone session (no Lesson 1), hand out the Reference Findings Sheet at the start instead of collecting peer reports, and skip the peer-swap logistics in the setup checklist.

UDL note: findings-to-control mapping table removes ambiguity in open-ended remediation planning; written/recorded choice remains for the final report; the custom rule template scaffolds the open-ended design task. Companion UDL worksheet available on request.

This lesson pair maps to OWASP GenAI/LLM Top 10 (2026): LLM01 Prompt Injection, LLM02 Sensitive Information Disclosure, and LLM08 Hidden Context Exposure. The same sandbox and gateway pattern extends to other Top 10 categories — LLM06 Unbounded Consumption and LLM10 Improper Output Handling are natural next lessons using this same architecture — available as a follow-on lesson series.

Source & attribution: lab environment and gateway adapted from github.com/SixFiveMil/Securing-AI. Companion to Lesson 1 (AI Red Team Engagement). First presented at the NCyTE Center webinar "Teaching AI Security: Hands-On LLM Hardening with Docker Desktop and Security Gateways," August 21, 2026.

Field	Details
Course level	Community college / intro–intermediate cybersecurity
Format	Single class session, in-person or virtual
Estimated time	75–90 minutes
Group size	Individual or pairs
Cost	\$0 — fully local; no cloud API keys, accounts, or spend required
Prerequisites	Completion of Lesson 1 (AI Red Team Engagement), or the reference findings sheet below if run standalone

Field	Details
Tools required	Docker Desktop, any web browser
Source materials	github.com/SixFiveMil/Securing-AI (Modelfiles, gateway, filter rules, setup commands)
Pairs with	Lesson 1 — AI Red Team Engagement (Attack, Assess, and Recommend)

Reading this table: the severity shown for each finding is the `vulnerable_bot` / `hardened_bot` pattern from Lesson 1's engagement (Conditions 1–2, gateway off), not a Condition-3-specific rating. What you are actually remediating is the upper end of that range — the fact that the finding was still reachable at all against `hardened_bot` with the gateway on (Condition 3) is what makes it an open finding for this lesson. If you are running Lesson 2 standalone, treat every row below as "still open against Condition 3" by definition; that is why it is on this sheet.

Finding	Severity	Root Cause
Direct Injection	Critical (<code>vulnerable_bot</code>) / High (<code>hardened_bot</code>)	Neither the system prompt nor the default gateway rules reliably block a direct override attempt phrased differently than the examples.
Roleplay / Hypothetical	Critical / High	Framing the request as fiction can bypass both the system prompt and simple keyword-based ingress filtering.
Obfuscation / Encoding	Critical / High	Encoded, spelled-out, or reformatted payloads are not recognized by exact-match egress checks.
Authority Claim	High / Medium–High	A persona claim ("I'm an IT administrator...") worded differently than the default blacklist entries slips past ingress filtering.

Time	Activity	Outcome(s)
0–5 min	Hook: read the follow-up client email. Ask: "What's the difference between saying it's fixed and proving it's fixed?" Share today's outcomes.	2
5–15 min	Mini-lecture: mapping finding types to remediation levers — model instructions vs. gateway filter rules vs. both — and how <code>filter_rules.py</code> implements input/output filtering.	1
15–25 min	Guided setup: receive findings (peer report or reference sheet); confirm the stack is running; review the gateway's default rule set together.	1
25–60 min	Core lab: for each finding, choose a remediation lever, implement it, and re-run the original attack to retest. Complete the Before/After Validation Matrix. Checkpoint at ~40 min.	1, 2
60–70 min	Design and implement one new custom filter rule addressing a gap not covered out of the box; justify it in writing.	3
70–80 min	Draft + share the Implementation & Validation Report (written or recorded). Pair/class share: "one finding, one fix, one piece of evidence."	2, 3

Finding	Original Severity	Remediation Lever (Prompt / Filter / Both)	Control Implemented	Retest Result (Fixed / Partial / Open)	Benign Sanity Check (Pass/Fail)
Direct Injection					
Roleplay / Hypothetical					
Obfuscation / Encoding					
Authority Claim					

Component	Meets expectations	Partial	Not yet
Remediation mapping (30%)	Every finding is mapped to a specific, correctly-reasoned remediation lever (prompt, filter, or both).	Most findings mapped; 1–2 mismatched or vague.	Mapping missing or not tied to findings.
Retest evidence (35%)	Each finding is empirically retested with clear before/after evidence.	Most findings retested; some evidence incomplete.	Retesting missing or not evidenced.
Custom filter rule (25%)	New rule is specific, correctly scoped, and clearly justified against a real gap.	Rule present but vague or loosely justified.	Rule missing or doesn't address an actual gap.
Residual risk note (10%)	Honestly identifies at least one remaining gap or limitation.	Note present but generic.	Note missing or claims zero residual risk.